

Integrated MSc(IT)

UNIT – II

Software Development An Agile Approach

What is Agile?



- Agile is the ability to create and respond to change.
- It is a way of dealing with, and ultimately succeeding in, an uncertain and turbulent environment.
- “AGILE” word represented the adaptiveness and response to change which was so important to their approach.
- It works by breaking projects down into little bits of user functionality called user stories
 - prioritizing them
 - continuously delivering them in short two week cycles called *iterations*.

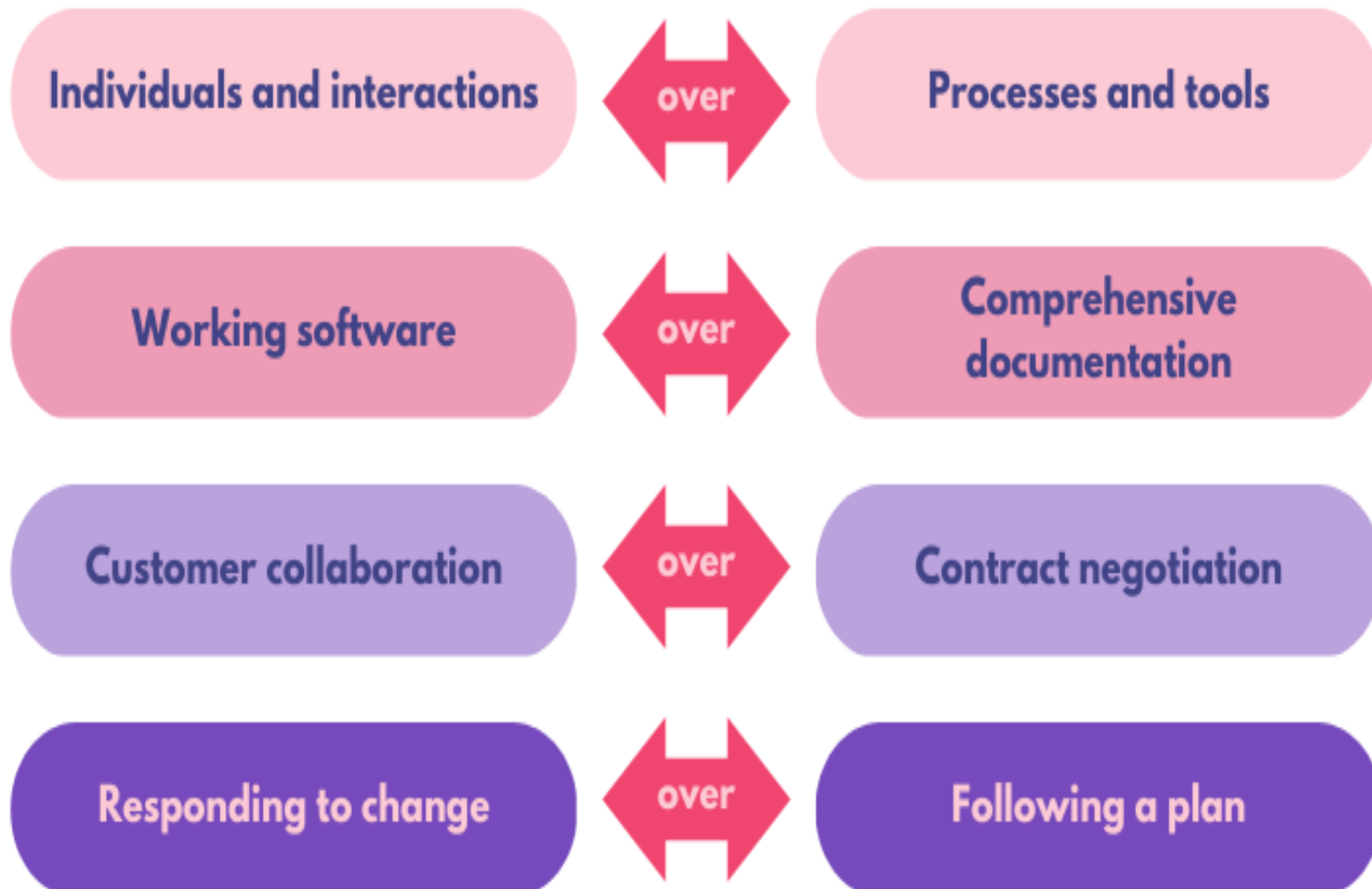
Introduction: Agile Methodology

- Agile software development is a collection of frameworks and practices that emphasize:
 - **Collaboration:** Working closely with stakeholders and customers to ensure releases deliver meaningful improvements
 - **Adaptability:** Being able to respond to changes in requirements, priorities, or market conditions
 - **Continuous Improvement:** Delivering value in small increments and implementing continuous feedback
 - **Quality:** Placing a strong emphasis on testing, quality assurance, and continuous improvement

Introduction: Agile Methodology

- **Agile Software Development** refers to:
 - Software Development Methodologies
 - Centered Round The Idea Of Iterative Development
 - requirements and solutions evolve through collaboration between self-organizing cross-functional teams
 - enables teams to deliver value faster, with greater quality and predictablity, and greater aptitude to respond to change.

4 Agile Manifesto values



Agile Methodology Features



Promote a disciplined project management process



Encourages frequent inspection and adaptation



A leadership that encourages teamwork, self-organization and accountability

Agile Methodology Features



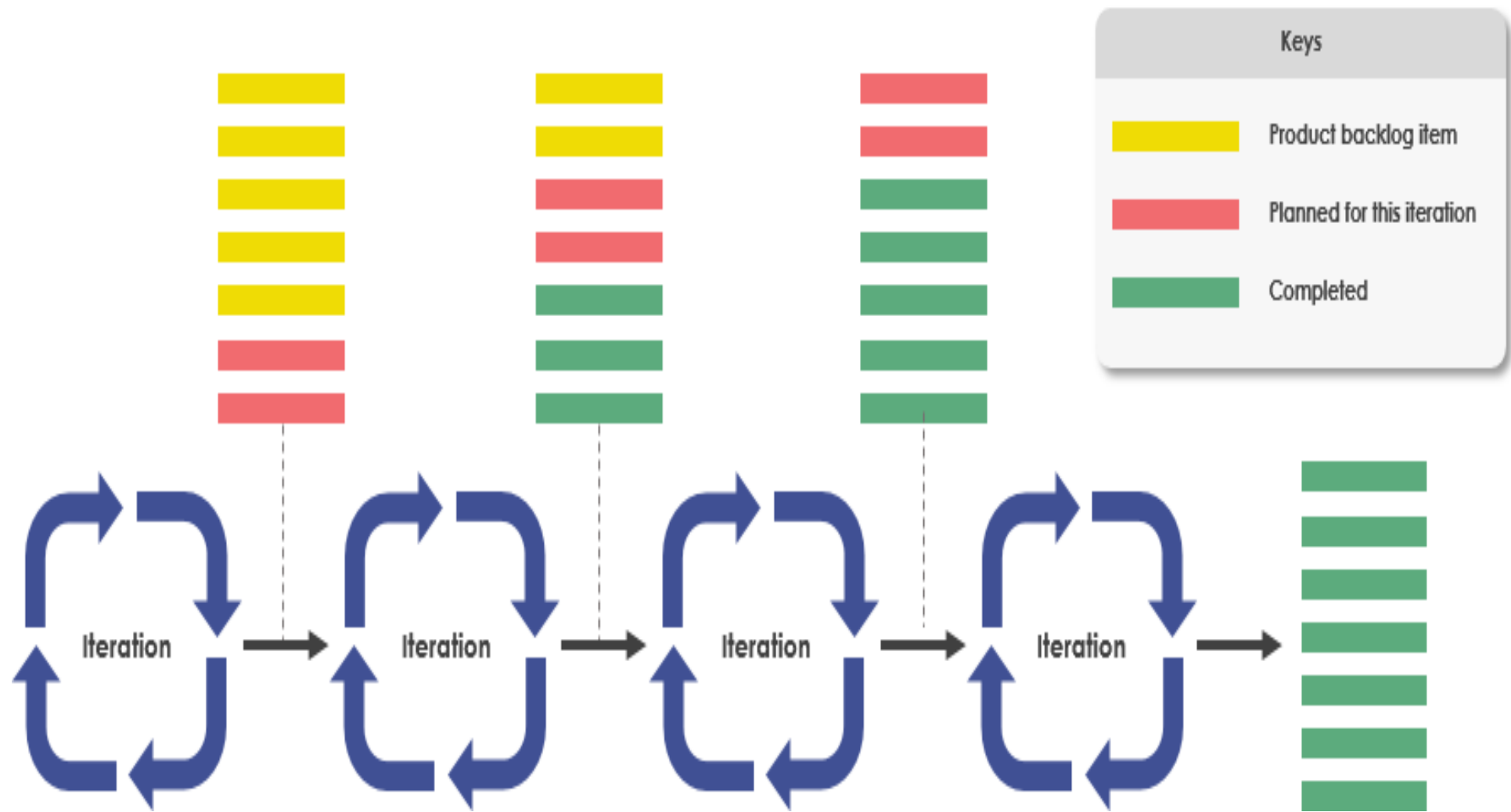
A set of engineering best practices intended to allow for rapid delivery



High quality software



A business approach that aligns development with customer needs and company goals.



Agile Process

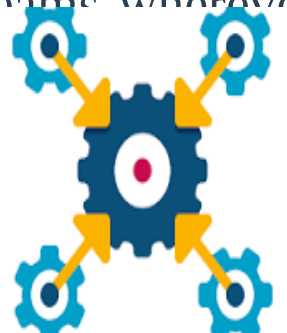
- "Agile process model" refers to a software development approach based on **iterative development**.
- Agile model believes that every project needs to be handled differently and the existing methods need to be **tailored to best suit the project requirements**.
- Agile methods **break tasks into smaller iterations**, or parts do not directly involve long term planning.
- **The project scope and requirements** are laid down at the beginning of the development process.
- Plans regarding the **number of iterations, the duration and the scope of each iteration are clearly defined** in advance.
- Each iteration is considered as a short time "**frame**" in the Agile process model, which typically lasts from one to four weeks.
- The division of the entire project into smaller parts helps to **minimize the project risk** and to reduce the **overall project delivery time requirements**.
- Each iteration involves a team working through a full software development life cycle including planning, requirements analysis, design, coding, and testing before a working product is demonstrated to the client.

Agile Process Characteristics

- It is difficult to predict in advance which **software requirements will persist** and which will **change**.
- It is equally difficult to **predict how customer priorities will change** as the project proceeds.
- For many types of software, design and construction are **interleaved**. That is, both activities should be performed in tandem so that design models are proven as they are created.
- It is difficult to **predict how much design is necessary** before construction is used to prove the design.
- Analysis, design, construction, and testing are not as **predictable** (from a planning point of view) as we might like.

What is Scrum?

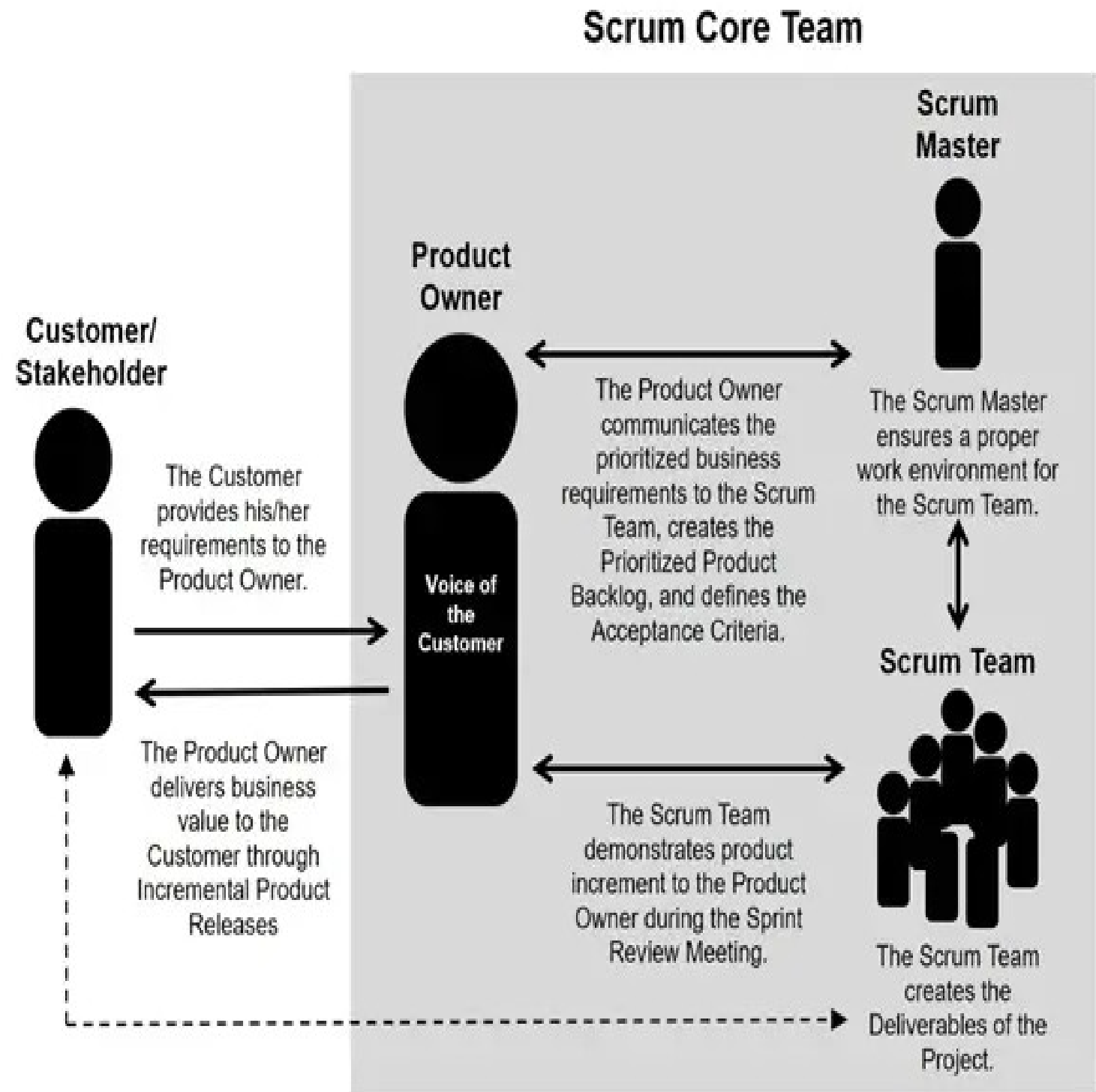
- Scrum is a project management framework that facilitates team collaboration on complex product and software development projects.
- The scrum methodology emphasizes teamwork in project management.
- It stresses accountability and iterative progress toward a well-defined goal.
- Scrum is part of agile software development and teams practice agile.
- **The name comes from the sport of rugby, where scrum is a formation where everyone plays a specific role, but everyone is working towards a quick adoption of strategies.**
- The necessary collaboration required for a successful scrum is facilitated by Project Manager, online and project management software that connects scrum teams wherever they work.



What is Scrum?

- Scrum is the type of Agile framework.
- *Agile is a set of principles and scrum is a framework for getting work done*
- It is a framework within which people can address complex adaptive problem while productivity and creativity of delivering product is at highest possible values.
- Scrum uses Iterative process.
- Silent features of Scrum are:
 - Scrum is light-weighted framework
 - Scrum emphasizes self-organization
 - Scrum is simple to understand
 - Scrum framework help the team to work together

Roles in Scrum



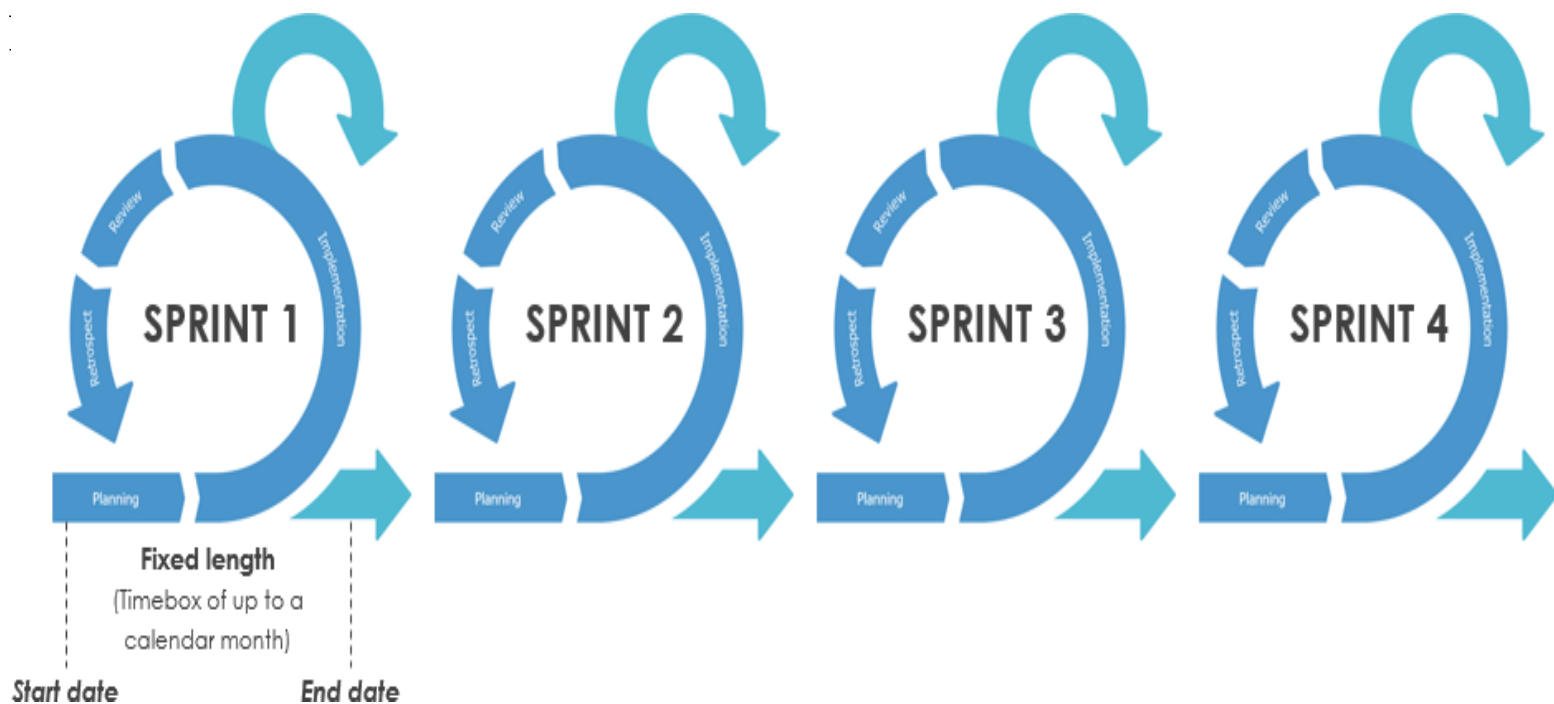
Key Roles	Key Values	Key Deliverables
Product Owner:	Accountable for maximizing the value of the product	- Clearly expressing the Product Backlog items - Ordering the items by Priority Order - Optimizing the value of the work - Ensuring that the PBIs are visible, transparent and clear to all - Ensuring the Development Team understands the items listed
Scrum Master:	Servant Leader who is responsible for promoting and supporting SCRUM as defined in the SCRUM Guide	- Ensuring the goals, scope, and product are understood by all - Removing impediments from the project - Product release planning - Facilitating SCRUM Events - Planning SCRUM Implementations
Development Team: Developers Technical Analyst Business Analyst Solution Architect IT Operations	Self-Organizing Team that is responsible to get the work completed.	- Develop solution and build out solution - Update system of record (ADO) project status - Documentation (Help Guides, FAQs, Test Scripts, Presentations) - Documentation Created for Architecture Review - Documentation for Evergreen Support identified, created and logged - Fully testing functionality for every increment and release

What is Sprint?

- A sprint is a short, time-boxed period when a scrum team works to complete a set amount of work.
- Sprints are at the very heart of scrum and agile methodologies, and getting sprints right will help your agile team ship better software with fewer headaches.
- With scrum, a product is built in a series of iterations called sprints that break down big, complex projects into bite-sized pieces.
- Sprints help teams follow the agile principle of "delivering working software frequently," as well as live the agile value of "responding to change over following a plan."

What is Sprint?

- A Sprint Planning Meeting occurs before the start of every sprint.
- That meeting defines what set of items could be developed and delivered in the upcoming sprint.
- Every sprint has a predefined goal and is meant to be accomplished in such a way that the sprint would culminate with a potentially shippable product increment.
- During every planning meeting, a Sprint Backlog is created that has a subset of items from the original product backlog.



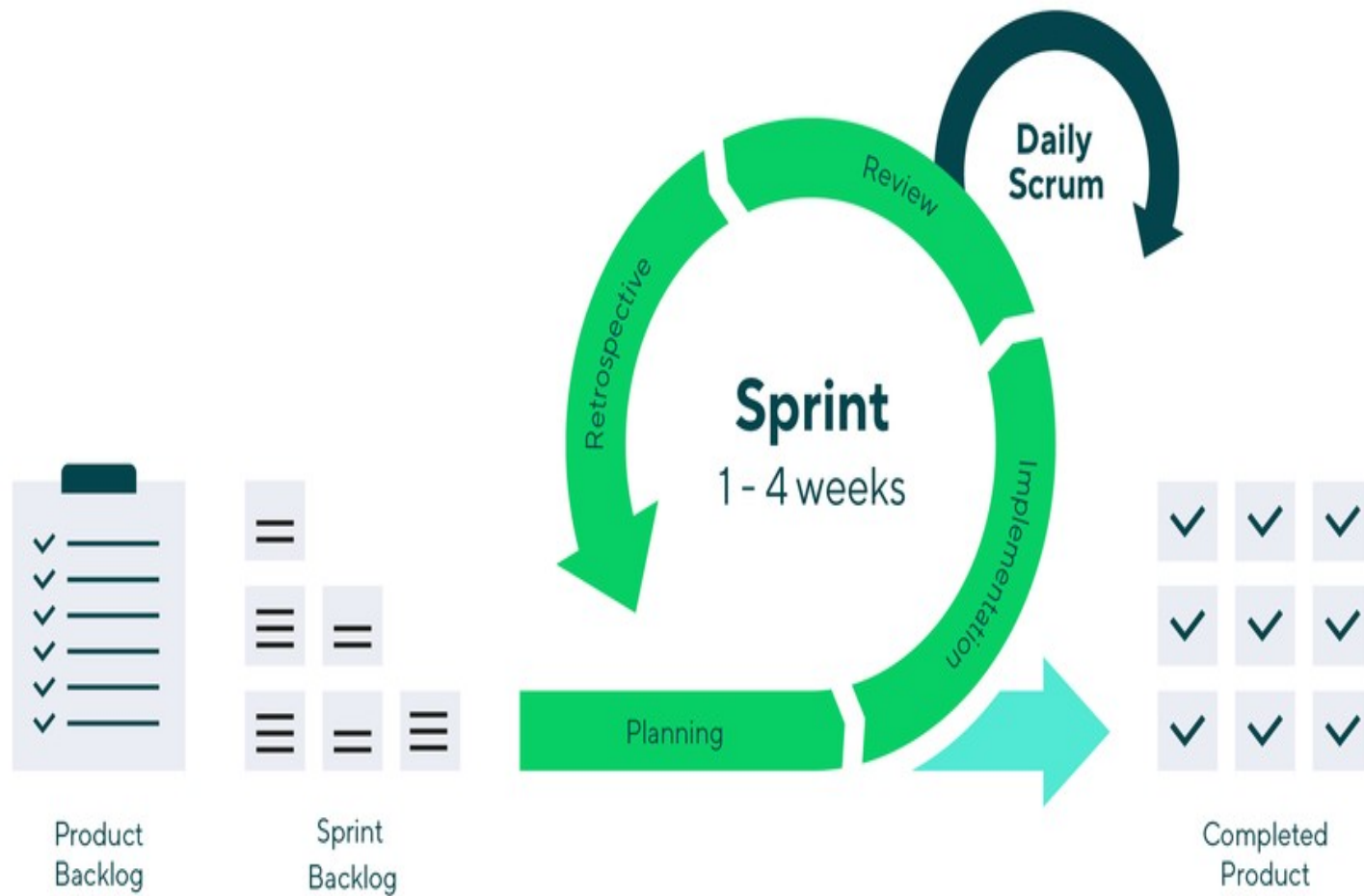
Sprint Workflow And Process

- **Backlog:**
 - It is a list of tasks need to do before the product is ready.
 - The product owner makes this list, and after that they gives it to the scrum master and team.
 - It is made up of user stories, all about what features and requirement list will gave the client.
- **Sprint Planning:**
 - In these team will focus on to the top things which are needs in user stories and decides what can be delivered in the sprint after discussing.
 - Sprint Planning means these meeting is usually done before the start of every sprint.
 - During this meeting, all the team members select some set of items (also called User Stories) from the Product Backlog.
 - These items are collectively termed Sprint Backlog and are the ones that will be developed during this sprint.

Sprint Workflow And Process

- **Sprint Backlog**
 - Entire team agrees on this list of tasks to be completed by development team during the sprint.
- **Sprint**
 - Sprint is basically the timeframe in which the work must be completed, like is around 30 days.
 - After the end of each sprint, all the team members present their work to the clients/stakeholders.
 - The iteration product is operated in a real-world organization environment.
 - In this meeting, the team gathers review and feedback from the clients and tend to rectify the changes, if any.
- **Daily Scrum**
 - The team will lead by the Scrum Master which collects every day for small meetings.
 - The meetings typically last 15 mins and the discussions revolve around all team members.
 - Questions discussed are:
 - What they have done?
 - What are they currently working on?
 - Any problems which they are facing during the work.

Sprint Workflow And Process



Sprint Workflow And Process

Sprint Planning:

- Using the product backlog, teams start with the highest priority items and determine how to achieve this objective.
- A good tip when sprint planning is to do the due diligence and only start with items that are ready.
- Also, remember that planning is a short process, so don't get bogged down in the details.
- Simply get to work on meeting the objectives and keep the plan collaborative.
- The team should also ask the product owner and stakeholder questions.

Sprint Workflow And Process

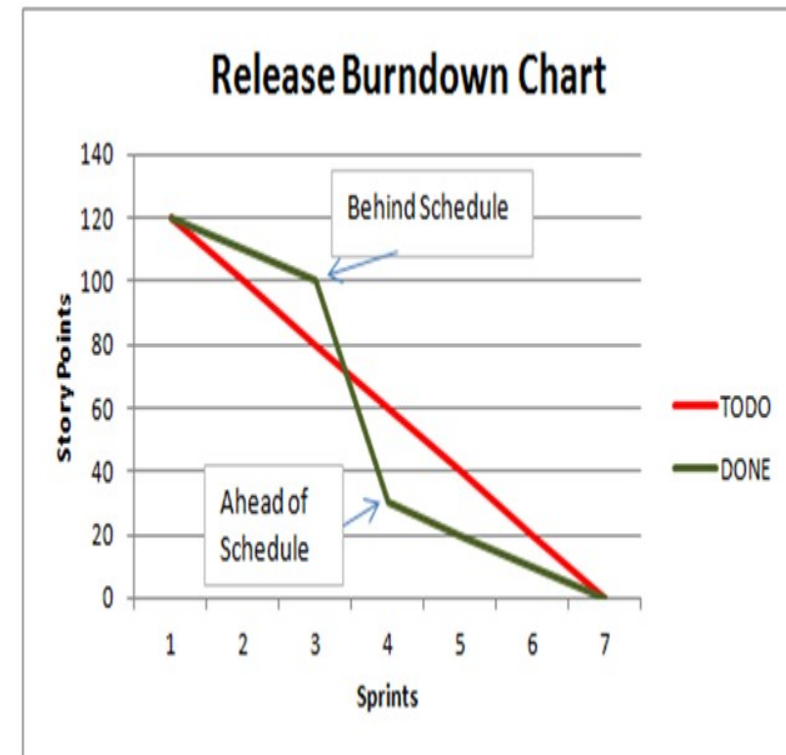
Daily Scrum Meeting

- These are 15-minute meetings where everybody in the scrum team talks about the tasks they'll be working on during the day and share any roadblocks or difficulties they're facing.
- There's no need to make this daily scrum meeting longer, as there are other meetings such as sprint reviews and sprint retrospectives to explore more complex topics.

Sprint Workflow And Process

Burnt Down Chart

- A burndown chart is a tool used by Agile teams to gather information about both the work they have completed on a project and the work that is yet to be done within a given time period.
- Agile teams use this simple, visual tool to determine how their project is progressing in the prescribed time and how much has been completed during each iteration.
- Burndown chart is a major parameter used in agile software development and scrum to detect how much work remains to be completed.
- It is the graphical representation of showing the left-out portion of the task versus time.



Sprint Workflow And Process

User Stories

- A user story is the smallest unit of work in an agile framework. It's an end goal, not a feature, expressed from the software user's perspective.
- A user story is an informal, general explanation of a software feature written from the perspective of the end user or customer.
- The purpose of a user story is to articulate how a piece of work will deliver a particular value back to the customer.
- They are short, simple descriptions of a feature or functionality from the perspective of a user.
- User stories are used to capture requirements in an agile project and help the development team understand the needs and expectations of the users.



As a <role>
I want <goal>
so that <benefit>

Acceptance criteria:
(Conditions of Satisfaction)

...
...

As an Account Manager
I want a sales report of my account
to be sent to my inbox daily
So that I can monitor the sales
progress of my customer portfolio

Acceptance criteria:

1. The report is sent daily to my inbox
2. The report contains the following sales details: ...
3. The report is in csv format.

Sprint Workflow And Process

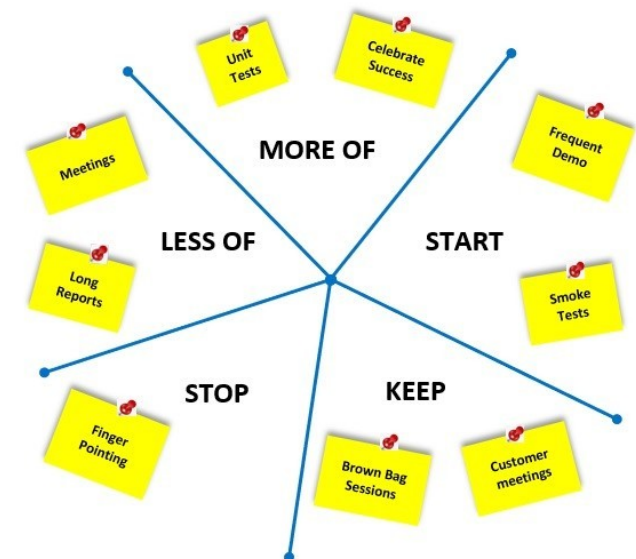


Sprint Workflow And Process

Retrospective

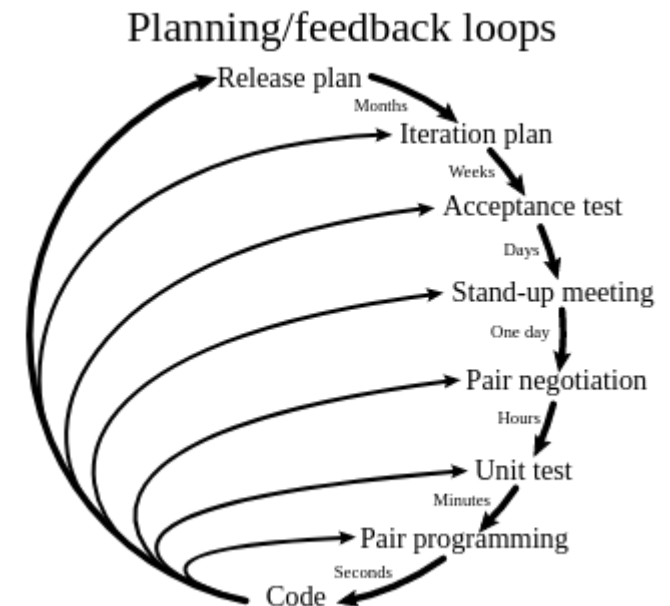
- An agile retrospective is a meeting that happens at the end of an agile development iteration.
- Team members share their highlights, challenges, and learnings from the last cycle.
- This helps teams apply these lessons later — easing up future work.
- Agile retrospectives (also called scrum or sprint retrospectives) are closely tied to scrum methodology.
- Retrospective meetings originated as one of the official scrum ceremonies.
- In scrum, agile retrospectives include the entire scrum team (comprised of a scrum master, product owner, and developers) and often product managers as well.

RETROSPECTIVES FOR FEEDBACK & LEARNING



Extreme Programming (XP)

- Extreme programming (XP) is one of the most important software development frameworks of Agile models.
- It is used to improve software quality and responsiveness to customer requirements.
- XP is quite helpful where there are frequent changes in the requirement and these changes can be incorporated easily.



Extreme Programming (XP)

- Each iteration of this model has the following phases:
- Iteration Planning: In this phase, the requirement is understood and drafted as stories and sub-tasks.
- Design: This phase takes care of designing the implementation.
- Coding or Implementation: The requirement is coded.
- Testing: The implementation is rigorously tested for defects
- Customers and developers work together to decide how to group stories.

Extreme Programming (XP)

- The XP team orders the stories that will be developed in one of three ways:
- All stories will be implemented immediately (within a few weeks)
- The stories with highest value will be moved up in the schedule and implemented first
- The riskiest stories will be moved up in the schedule and implemented first.
- After the first project release (also called a software increment) has been delivered, the XP team computes project velocity.

Extreme Programming (XP): Practices

- **Code Review:** Code review detects and corrects errors efficiently. It suggests pair programming as coding and reviewing of written code carried out by a pair of programmers who switch their work between them every hour.
- **Testing:** Testing code helps to remove errors and improves its reliability. XP suggests test-driven development (TDD) to continually write and execute test cases. In the TDD approach, test cases are written even before any code is written.
- **Incremental Development:** Incremental development is very good because customer feedback is gained and based on this development team comes up with new increments every few days after each iteration.

Extreme Programming (XP): Practices

- **Simplicity:** Simplicity makes it easier to develop good-quality code as well as to test and debug it.
- **Design:** Good quality design is important to develop good quality software. So, everybody should design daily.
- **Integration Testing:** Integration Testing helps to identify bugs at the interfaces of different functionalities. Extreme programming suggests that the developers should achieve continuous integration by building and performing integration testing several times a day.

Extreme Programming (XP): Values and Principles

- **Coding:** The concept of coding which is used in the XP model is slightly different from traditional coding. Here, the coding activity includes drawing diagrams (modeling) that will be transformed into code, scripting a web-based system, and choosing among several alternative solutions.
- **Testing:** The XP model gives high importance to testing and considers it to be the primary factor in developing fault-free software.
- **Listening:** The developers need to carefully listen to the customers if they have to develop good quality software. Sometimes programmers may not have the depth knowledge of the system to be developed. So, the programmers should understand properly the functionality of the system and they have to listen to the customers.

Extreme Programming (XP): Values and Principles

- **Designing:** Without a proper design, a system implementation becomes too complex, and very difficult to understand the solution, thus making maintenance expensive. A good design results elimination of complex dependencies within a system. So, effective use of suitable design is emphasized.
- **Feedback:** One of the most important aspects of the XP model is to gain feedback to understand the exact customer needs. Frequent contact with the customer makes the development effective.
- **Simplicity:** The main principle of the XP model is to develop a simple system that will work efficiently in the present time, rather than trying to build something that would take time and may never be used. It focuses on some specific features that are immediately needed, rather than engaging time and effort on speculations of future requirements.

Extreme Programming (XP): Values and Principles

- **Pair Programming:** XP encourages pair programming where two developers work together at the same workstation. This approach helps in knowledge sharing, reduces errors, and improves code quality.
- **Continuous Integration:** In XP, developers integrate their code into a shared repository several times a day. This helps to detect and resolve integration issues early on in the development process.
- **Refactoring:** XP encourages refactoring, which is the process of restructuring existing code to make it more efficient and maintainable. Refactoring helps to keep the codebase clean, organized, and easy to understand.

Extreme Programming (XP): Values and Principles

- **Collective Code Ownership:** In XP, there is no individual ownership of code. Instead, the entire team is responsible for the codebase. This approach ensures that all team members have a sense of ownership and responsibility towards the code.
- **Planning Game:** XP follows a planning game, where the customer and the development team collaborate to prioritize and plan development tasks. This approach helps to ensure that the team is working on the most important features and delivers value to the customer.
- **On-site Customer:** XP requires an on-site customer who works closely with the development team throughout the project. This approach helps to ensure that the customer's needs are understood and met, and also facilitates communication and feedback.

Extreme Programming (XP): Applications

- **Small Projects:** The XP model is very useful in small projects consisting of small teams as face-to-face meeting is easier to achieve.
- **Projects involving new technology or Research projects:** This type of project faces changing requirements rapidly and technical problems. So XP model is used to complete this type of project.
- **Web development projects:** The XP model is well-suited for web development projects as the development process is iterative and requires frequent testing to ensure the system meets the requirements.
- **Collaborative projects:** The XP model is useful for collaborative projects that require close collaboration between the development team and the customer.
- **Projects with tight deadlines:** The XP model can be used in projects that have a tight deadline, as it emphasizes simplicity and iterative development.
- **Projects with rapidly changing requirements:** The XP model is designed to handle rapidly changing requirements, making it suitable for projects where requirements may change frequently.
- **Projects where quality is a high priority:** The XP model places a strong emphasis on testing and quality assurance, making it a suitable approach for projects where quality is a high priority.

Extreme Programming (XP): Life Cycle

Life Cycle of Extreme Programming (XP)

